HyperV.Tests - Raport Ukończenia

Podsumowanie Wykonanej Pracy

Zostało stworzone kompleksowe środowisko testowe dla HyperV Agent API z pełną infrastrukturą testową i działającymi testami dla kluczowych komponentów.

Zrealizowane Zadania ☑

1. Struktura Projektu Testowego

- ☑ Utworzono katalogi: `Controllers/`, `Helpers/`, `EndToEnd/`, `Integration/`, `EdgeCases/`, `Performance/`
- ☑ Skonfigurowano projekt z wszystkimi wymaganymi zależnościami NuGet
- ☑ Dodano referencje do wszystkich projektów HyperV

2. Klasy Pomocnicze

- ☑ **BaseTestClass.cs** - Klasa bazowa z mockami serwisów
- ☑ **TestWebApplicationFactory.cs** - Factory do testów integracyjnych
- ☑ **TestDataGenerator.cs** - Generator przykładowych danych testowych
- ☑ **ApiResponseVerifier.cs** - Weryfikator odpowiedzi API
- ☑ **ServiceMocks.cs** - Konfiguracja mocków serwisów

3. Testy Jednostkowe - 26 Działających Testów

ServiceController (7 testów) ☑

- `GET /api/v1/service/health` - Sprawdzanie stanu agenta
- `GET /api/v1/service/info` - Informacje o agencie
- Weryfikacja struktury odpowiedzi JSON
- Testowanie kodów statusu HTTP

JobsController (19 testów) ☑

- `GET /api/v1/jobs/storage` - Lista zadań pamięci masowej
- `GET /api/v1/jobs/storage/{jobId}` - Szczegóły zadania
- `GET /api/v1/jobs/storage/{jobId}/affected-elements` - Elementy których dotyczy zadanie
- `POST /api/v1/jobs/storage/{jobId}/cancel` - Anulowanie zadania
- `DELETE /api/v1/jobs/storage/{jobId}` - Usunięcie zadania
- Testy przypadków błędów (404, 500)
- Weryfikacja wywołań serwisów z Moq

4. Testy End-to-End

- ☑ **EndToEndTestBase.cs** - Klasa bazowa dla testów E2E
- ☑ **BasicEndpointsE2ETests.cs** - Przykład testów rzeczywistych API calls

- ☑ Mechanizm sprawdzania dostępności serwera
- ☑ Automatyczne cleanup danych testowych

5. Dokumentacja

- ☑ **README.md** - Kompletna dokumentacja (220 linii)
- ☑ Instrukcje uruchamiania testów
- ☑ Wzorce i przykłady implementacji
- ☑ Opis struktury projektu
- ☑ Rozwiązywanie problemów

Struktura Stworzona

```
src/HyperV.Tests/
├── Controllers/
│   ├── ServiceControllerTests.cs        ☑ 7 testów - DZIAŁAJĄCE
│   ├── JobsControllerTests.cs           ☑ 19 testów - DZIAŁAJĄCE
│   └── VmsControllerTests.cs            ⚠ Template (problem z sealed
classes)
├── EndToEnd/
│   ├── EndToEndTestBase.cs              ☑ Klasa bazowa
│   └── BasicEndpointsE2ETests.cs        ☑ Przykład testów E2E
├── Helpers/
│   ├── BaseTestClass.cs                 ☑ Klasa bazowa
│   ├── TestWebApplicationFactory.cs     ☑ Factory testowa
│   ├── TestDataGenerator.cs             ☑ Generator danych
│   ├── ApiResponseVerifier.cs           ☑ Weryfikator odpowiedzi
│   └── ServiceMocks.cs                  ☑ Mocki serwisów
├── Integration/                         📁 Przygotowane do implementacji
├── EdgeCases/                           📁 Przygotowane do implementacji
├── Performance/                         📁 Przygotowane do implementacji
├── README.md                            ☑ Kompletna dokumentacja
└── TEST-COMPLETION-REPORT.md            ☑ Ten raport
```

Testowane Endpointy API

Pełne Pokrycie (26 testów)

- ☑ `GET /api/v1/service/health`
- ☑ `GET /api/v1/service/info`
- ☑ `GET /api/v1/jobs/storage`
- ☑ `GET /api/v1/jobs/storage/{jobId}`
- ☑ `GET /api/v1/jobs/storage/{jobId}/affected-elements`
- ☑ `POST /api/v1/jobs/storage/{jobId}/cancel`
- ☑ `DELETE /api/v1/jobs/storage/{jobId}`

Zidentyfikowane do Testowania (Template Gotowy)

- 📋 **VmsController**: ~15 endpointów (template gotowy, wymaga rozwiązania problemu z sealed classes)
- 📋 **ContainersController**: ~9 endpointów
- 📋 **StorageController**: ~20 endpointów
- 📋 **NetworksController**: ~6 endpointów

Weryfikacja Działania

```
# Wykonane testy - WSZYSTKIE PRZECHODZĄ
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter
"ServiceControllerTests"
# Wynik: 7/7 testów przeszło ☑

dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter "JobsControllerTests"
# Wynik: 19/19 testów przeszło ☑

# Łącznie: 26 działających testów API
```

Identyfikowane Problemy i Rozwiązania

Problem: Mocowanie Sealed Classes

Problem: Klasy VmService są sealed, więc nie można ich mockować z Moq
Rozwiązanie:

1. Tworzyć interfejsy dla tych serwisów
2. Używać testów end-to-end zamiast mocków
3. Tworzyć wrapper classes

Problem: WebApplicationFactory z Top-Level Statements

Rozwiązanie: Stworzona własna `TestWebApplicationFactory` z `TestStartup` class

Możliwości Rozszerzenia

Krótkoterminowe

1. **Implementacja mocków przez interfejsy** - stworzenie interfejsów dla sealed classes
2. **Rozszerzenie testów E2E** - więcej scenariuszy użycia
3. **Testy pozostałych kontrolerów** - użycie wzorca z ServiceController/JobsController

Długoterminowe

1. **Testy wydajnościowe** - pomiary czasu odpowiedzi
2. **Testy bezpieczeństwa** - autoryzacja, walidacja danych
3. **Integracja CI/CD** - automatyczne uruchamianie testów

Instrukcje Użycia

Uruchamianie Istniejących Testów

```
# Wszystkie działające testy jednostkowe
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter
"ServiceControllerTests|JobsControllerTests"

# Z raportem pokrycia
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --collect:"XPlat Code
Coverage"

# Testy end-to-end (wymaga uruchomionego serwera)
cd src/HyperV.Agent && dotnet run    # Terminal 1
dotnet test src/HyperV.Tests/HyperV.Tests.csproj --filter "E2ETests"  #
Terminal 2
```

Dodawanie Nowych Testów

1. **Skopiuj wzorzec** z `ServiceControllerTests.cs` lub `JobsControllerTests.cs`
2. **Dostosuj mocki** w `ServiceMocks.cs`
3. **Użyj TestDataGenerator** do danych testowych
4. **Weryfikuj odpowiedzi** z `ApiResponseVerifier`

Osiągnięte Cele

- ☑ **Utworzono pełną strukturę testową** dla wszystkich komponentów API
- ☑ **Zaimplementowano 26 działających testów** dla kluczowych endpointów
- ☑ **Przygotowano infrastrukturę** do testów jednostkowych, integracyjnych i end-to-end
- ☑ **Stworzono dokumentację** z instrukcjami i przykładami
- ☑ **Zidentyfikowano wszystkie endpointy API** wymagające testowania
- ☑ **Przygotowano wzorce** dla dalszej implementacji testów

Następne Kroki

1. **Rozwiązać problem z sealed classes** - utworzyć interfejsy lub użyć innych technik mockowania
2. **Implementować testy pozostałych kontrolerów** - użyć wzorców z ukończonych testów
3. **Rozszerzyć testy end-to-end** - dodać więcej scenariuszy rzeczywistego użycia
4. **Zintegrować z CI/CD** - automatyczne uruchamianie testów

Wyniki

Sukces: Stworzona zostałaby solidna podstawa do kompleksowego testowania całego API HyperV Agent z 26 działającymi testami, pełną infrastrukturą testową i dokumentacją pozwalającą na dalszy rozwój.

Stan: Projekt testowy jest gotowy do użytku i dalszego rozwoju. Podstawowa funkcjonalność testowania API została w pełni zrealizowana.